Advanced Machine Learning

DATA 642

Lecture 4 — LASSO

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, and Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier.

Lasso

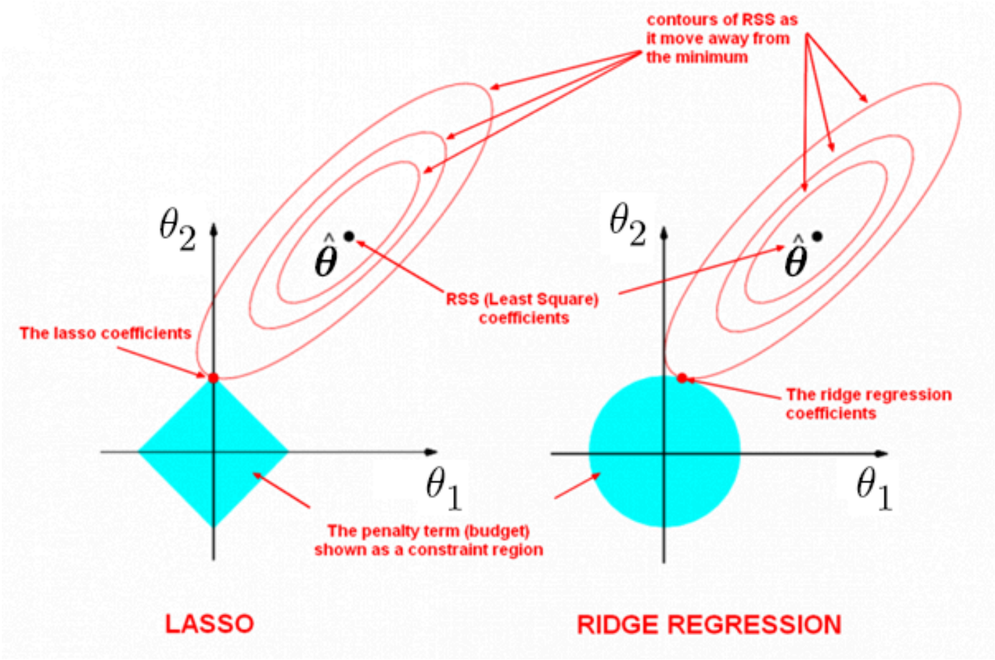
Motivation: The penalty term $\lambda \sum_{j=1}^{p} \beta_j^2$ will shrink all of the coefficients towards zero but it will not set any of them exactly to zero. This is not good for forcing **sparsity**.

Lasso is a recent alternative

$$\sum_{n=1}^{N}\Big(y_i - \theta_0 - \sum_{j=1}^{l} \theta_j x_j\Big)^2 + \lambda \sum_{j=1}^{l} |\theta_j|, \tag{1}$$

where $\sum_{j=1}^{l} |\theta_j|$ denotes the ℓ_1 norm of the vector $[\theta_1, \theta_2, \ldots, \theta_l]^\top$.

Geometric interpretation



As l increases, the multidimensional diamond has an increasing number of corners, and so it is highly likely that some coefficients will be set equal to zero. Hence, the lasso performs shrinkage and (effectively) subset selection.

The solution

The Lasso (Least Absolute Shrinkage and Selection Operator) problem aims to minimize the following objective function:

$$J(\boldsymbol{\theta}) = \frac{1}{2N}||\mathbf{y} - \mathbf{X}\boldsymbol{\theta}||_2^2 + \lambda||\boldsymbol{\theta}||_1 \tag{2}$$

Where:

- $\mathbf{y}$ is the vector of target values.
- $\mathbf{X}$ is the design matrix.
- $\boldsymbol{\theta}$ is the vector of coefficients.
- N is the number of samples.
- λ is the regularization parameter, controlling the strength of regularization.

The first term in the objective function represents the squared error loss, and the second term represents the ℓ_1-norm regularization penalty.

To find the solution for Lasso, we aim to minimize this objective function with respect to $\boldsymbol{\theta}$. However, the ℓ_1-norm regularization term makes the problem non-differentiable at $\theta_j = 0$ for $j = 1, \ldots, l$. Therefore, standard gradient-based optimization techniques cannot be directly applied.

One common approach to solve the Lasso problem is by using the sub-gradient method. The sub-gradient of the objective function at any point $\boldsymbol{\theta}$ is a vector that generalizes the notion of the gradient to non-differentiable functions. The sub-gradient equation for the Lasso problem is given by:

$$g(\boldsymbol{\theta}, \lambda) = -\frac{1}{N}\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) + \lambda \begin{bmatrix} \operatorname{sgn}(\theta_1) \\ \operatorname{sgn}(\theta_2) \\ \vdots \\ \operatorname{sgn}(\theta_l) \end{bmatrix} \tag{3}$$

Where $\text{sgn}(\theta_j)$ is the sign function defined as:

$$\text{sgn}(\theta_j) = \begin{cases} -1 & \text{if } \theta_j < 0 \\ \text{any value between } -1 \text{ and } 1 & \text{if } \theta_j = 0 \\ 1 & \text{if } \theta_j > 0 \end{cases} \tag{4}$$

Once we have the sub-gradient, we can use optimization algorithms such as gradient descent with this sub-gradient to iteratively update $\boldsymbol{\theta}$ until convergence, thus finding the optimal solution for the Lasso problem.

Comments

1. Lasso does **variance reduction**, **provides accurate predictions**, and is ideal for **variable selection**.

2. In many practical applications it is good to have a little bit of regularization. Ridge is always a good default but if you suspect that only a few features or variables are useful you should prefer Lasso.

3. If $\mathbf{X}$ has orthonormal columns, then Lasso receives a closed form solution which is given by

$$\hat{\theta}_j = \operatorname{sgn}(\hat{\theta}_{\mathrm{LS}})(|\hat{\theta}_{\mathrm{LS}}| - \lambda/2)_+ \tag{5}$$

   This is the soft thresholding operator, where $(\cdot)_+$ denotes the "positive part" of the respective argument; it is equal to the argument if this is nonnegative, and zero otherwise.

4. If $\mathbf{X}$ has orthonormal columns, then $\hat{\boldsymbol{\theta}}_R = \frac{1}{1+\lambda}\hat{\boldsymbol{\theta}}_{\mathrm{LS}}$.

Computational complexity

1. As with linear regression, we can perform ridge regression either by computing a closed-form equation or by performing gradient descent.

2. In the case where there are a large number of features/variables or too many training instances to fit in memory it may be better to use gradient descent.

3. Soft-thresholding is only one possibility for solving the Lasso problem. There are alternatives. For instance, approximate the ℓ_1 norm by a family of quadratic curves

$$\lim_{\epsilon \to 0} \sum_{i=0}^{l} \sqrt{\theta_i^2 + \epsilon} \tag{6}$$